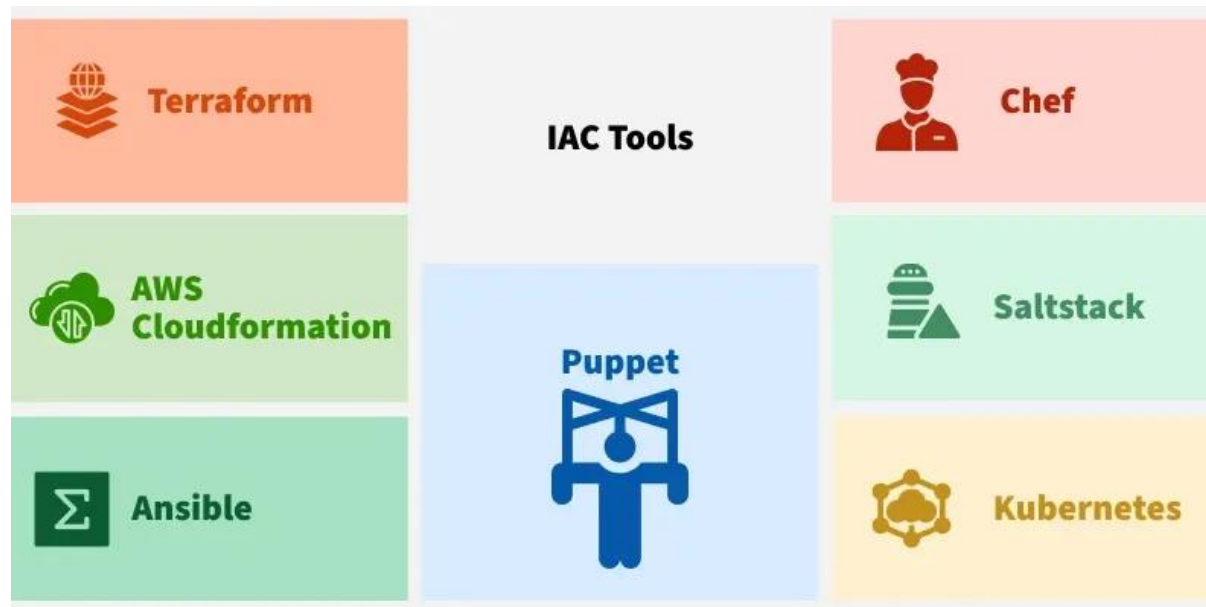




IaC – Terraform - 1

What is Infrastructure as Code – IaC?

- Infrastructure as Code (IaC) is a modern DevOps practice where servers, networks, and cloud resources are automatically provisioned and managed using code instead of manual configuration. It helps maintain consistency, speed, and reliability across deployments.
- It treats infrastructure components like software, allowing you to use development practices like version control, testing, and CI/CD pipelines.



What is Infrastructure as Code – IaC?

- An idempotent operation is one that can be applied multiple times without changing the result beyond the initial application. In IaC terms, this means that running your IaC script over and over will always result in the same defined state. **If the infrastructure already matches the desired state, the tool will simply do nothing.** This makes applying updates safe and predictable. It treats infrastructure components like software, allowing you to use development practices like version control, testing, and CI/CD pipelines.
- Mutable Infrastructure: Servers are provisioned and then updated, patched, and modified in place over their lifetime. This process can lead to configuration drift and makes servers fragile and unique.
- Immutable Infrastructure: When a change is needed, the existing server is not modified. Instead, a new server is provisioned from a fresh image with the desired changes, and the old one is terminated. This ensures a clean, predictable, and fully reproducible state.

What is Infrastructure as Code – IaC?

- IaC tools follow one of two main approaches to defining infrastructure: Declarative Approach or Imperative Approach
- Declarative:
 - Specifies the desired state (the "what") of the infrastructure.
 - The IaC tool determines and performs the actions needed to reach the desired state.
 - The tool tracks the current state of the infrastructure, simplifying updates and teardown.
 - Simplifies the process; users define what they want.
 - Automatically calculates and applies the necessary changes to match the new desired state.
 - Defining a resource in a Terraform file: `resource "aws_instance" "web" { ami = "ami-123" instance_type = "t2.micro" }`
- Imperative:
 - Details the exact steps or commands (the "how") to achieve the desired state.
 - Requires the user to execute commands in the correct sequence.
 - Does not inherently track state; the user is responsible for managing changes.
 - Demands detailed instructions; the user defines how to achieve the result.
 - The user must write a new script to figure out and apply the changes manually.
 - Writing a shell script: `#!/bin/bashaws ec2 run-instances \ --image-id ami-123 \ --instance-type t2.micro`

What is Infrastructure as Code – IaC?

- IaC is a fundamental enabler of DevOps practices by bridging the gap between development and operations:
 - Enabling CI/CD: IaC integrates seamlessly into Continuous Integration/Continuous Delivery (CI/CD) pipelines. A code change can trigger a pipeline that not only tests the application but also provisions or updates the necessary infrastructure to run it.
 - Creating Ephemeral Environments: Teams can instantly spin up new, production-like environments for development, testing, or user acceptance testing (UAT). This allows developers to test their code in a realistic setting before merging, catching bugs earlier.
 - Breaking Down Silos: When infrastructure is code, it lives in the same VCS as the application code. Developers and operations engineers can collaborate on the same files, review changes through pull requests, and gain a shared understanding of the entire system.

Popular IaC tools

Infrastructure Provisioning Tools

- These tools are primarily used to create, modify, and destroy the foundational infrastructure components like virtual machines, networks, and databases. They are almost always declarative:
 - Terraform: A widely-used, open-source tool by HashiCorp. Its key feature is being cloud-agnostic, supporting AWS, Azure, GCP, and many other providers. It uses a declarative language (HCL) and excels at managing complex, multi-cloud infrastructure.
 - AWS Cloud Formation: The native IaC tool for AWS. It allows you to define AWS resources in JSON or YAML templates. Its main strength is its deep integration with all AWS services.

Popular IaC tools

Configuration Management Tools

These tools specialize in configuring the software on existing servers. They install packages, manage configuration files, and ensure services are running.

- Ansible: A very popular open-source tool known for its simplicity and agentless architecture. It uses YAML "playbooks" that are easy to read and operates over SSH. It can be used for both imperative (tasks) and declarative (state) management.
- Puppet: A mature, agent-based tool that uses a declarative, model-driven approach. A central "Puppet Master" server manages the "Puppet Agents" on each machine, enforcing the desired state.

Popular IaC tools

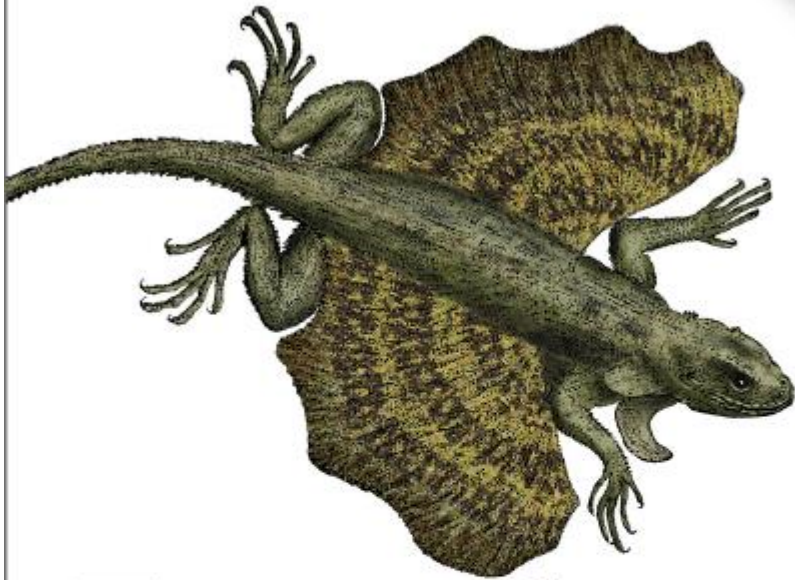
Container Orchestration Tools

While not strictly traditional IaC, these tools manage the infrastructure and lifecycle of containerized applications in a declarative way.

Kubernetes (K8s): The de-facto standard for container orchestration. Kubernetes automates the deployment, scaling, and management of containerized applications. Users define the desired state of the application (e.g., "run 3 replicas of this container and expose it on port 80") using YAML manifests.

O'REILLY®

Third
Edition



Terraform

Up & Running

Writing Infrastructure as Code

Yevgeniy Brikman

Introduction to Terraform

- Terraform is an open-source infrastructure as code tool that allows you to define, provision, and manage cloud infrastructure using a declarative configuration language.
- Unlike traditional approaches where infrastructure is manually configured through web consoles or imperative scripts, Terraform lets you describe your desired infrastructure state in code files, and the tool automatically determines the steps needed to achieve that state.
- The HashiCorp Configuration Language (HCL) is Terraform's native language for writing infrastructure definitions.
- HCL is declarative, meaning you specify what you want to create rather than how to create it. For example, instead of writing a script with sequential steps to create a server, configure its security settings, and add it to a load balancer, you simply declare that you want a server with certain properties, a security group with specific rules, and a load balancer that routes to that server. Terraform analyzes these declarations, builds a dependency graph, and executes the necessary API calls in the correct order to make your infrastructure match your code.

Installing Terraform

The easiest way to install Terraform is to use your operating system's package manager.

For example, on macOS, if you are a Homebrew user, you can run the following:

```
$ brew tap hashicorp/tap
```

```
$ brew install hashicorp/tap/terraform
```

On Windows, if you're a Chocolatey user, you can run the following:

```
$ choco install terraform
```

Alternatively, you can install Terraform manually by going to the Terraform home page, clicking the download link, selecting the appropriate package for your operating system, downloading the ZIP archive, and unzipping it into the directory where you want Terraform to be installed. The archive will extract a single binary called terraform, which you'll want to add to your PATH environment variable.

Installing Terraform

For Terraform to be able to make changes in your AWS account, you will need to set the AWS credentials for the IAM user you created earlier as the environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.

For example, here is how you can do it in a Unix/Linux/macOS terminal:

```
$ export AWS_ACCESS_KEY_ID=(your access key id)
```

```
$ export AWS_SECRET_ACCESS_KEY=(your secret access key)
```

And here is how you can do it in a Windows command terminal:

```
$ set AWS_ACCESS_KEY_ID=(your access key id)
```

```
$ set AWS_SECRET_ACCESS_KEY=(your secret access key)
```

Deploying a Single Server

- Terraform code is written in the HashiCorp Configuration Language (HCL) in files with the extension .tf.
- It is a declarative language, so your goal is to describe the infrastructure you want, and Terraform will figure out how to create it.
- Terraform can create infrastructure across a wide variety of platforms, or what it calls providers, including AWS, Azure, Google Cloud, DigitalOcean, and many others.
- The first step to using Terraform is typically to configure the provider(s) you want to use. Create an empty folder and put a file in it called main.tf that contains the following contents:

```
provider "aws" {  
  region = "us-east-2"  
}
```

This tells Terraform that you are going to be using AWS as your provider and that you want to deploy your infrastructure into the us-east-2 region.

Deploying a Single Server

For each type of provider, there are many different kinds of resources that you can create, such as servers, databases, and load balancers.

The general syntax for creating a resource in Terraform is as follows:

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
  [CONFIG ...]  
}
```

where PROVIDER is the name of a provider (e.g., aws), TYPE is the type of resource to create in that provider (e.g., instance), NAME is an identifier you can use throughout the Terraform code to refer to this resource (e.g., my_instance), and CONFIG consists of one or more arguments that are specific to that resource.

Deploying a Single Server

For example, to deploy a single (virtual) server in AWS, known as an EC2 Instance, use the `aws_instance` resource in `main.tf` as follows:

```
resource "aws_instance" "example" {  
  ami = "ami-0fb653ca2d3203ac1"  
  instance_type = "t2.micro"  
}
```

The `aws_instance` resource supports many different arguments, but for now, you only need to set the two required ones:

Ami: The Amazon Machine Image (AMI) to run on the EC2 Instance. You can find free and paid AMIs in the AWS Marketplace or create your own using tools such as Packer. The preceding code sets the `ami` parameter to the ID of an Ubuntu 20.04 AMI in `us-east-2`.

Deploying a Single Server

instance_type: The type of EC2 Instance to run. Each type of EC2 Instance provides a different amount of CPU, memory, disk space, and networking capacity. The EC2 Instance Types page lists all the available options. The preceding example uses t2.micro, which has one virtual CPU, 1 GB of memory, and is part of the AWS Free Tier.

In a terminal, go into the folder where you created main.tf and run the terraform init command:

```
$ terraform init
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using hashicorp/aws v4.19.0 from the shared cache directory

```
Terraform has been successfully initialized!
```


Deploying a Single Server

- The terraform binary contains the basic functionality for Terraform, but it does not come with the code for any of the providers (e.g., the AWS Provider, Azure provider, GCP provider, etc.), so when you're first starting to use Terraform, you need to run `terraform init` to tell Terraform to scan the code, figure out which providers you're using, and download the code for them.
- By default, the provider code will be downloaded into a `.terraform` folder, which is Terraform's scratch directory (you may want to add it to `.gitignore`).
- Terraform will also record information about the provider code it downloaded into a `terraform.lock.hcl` file (you'll learn more about this file). You'll see a few other uses for the `init` command and `.terraform` folder in later chapters.
- For now, just be aware that you need to run `init` anytime you start with new Terraform code and that it's safe to run `init` multiple times (the command is idempotent).

Deploying a Single Server

Now that you have the provider code downloaded, run the terraform plan command:

```
$ terraform plan
```

```
(...)
```

Terraform will perform the following actions:

```
    # aws_instance.example will be created
+ resource "aws_instance" "example" {
+   ami                = "ami-0fb653ca2d3203ac1"
+   arn                = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone   = (known after apply)
+   cpu_core_count      = (known after apply)
+   cpu_threads_per_core = (known after apply)
+   get_password_data   = false
+   host_id             = (known after apply)
+   id                  = (known after apply)
+   instance_state      = (known after apply)
+   instance_type       = "t2.micro"
+   ipv6_address_count  = (known after apply)
+   ipv6_addresses      = (known after apply)
+   key_name            = (known after apply)
+   (...)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Deploying a Single Server

- The plan command lets you see what Terraform will do before actually making any changes. This is a great way to sanity-check your code before unleashing it onto the world.
- The output of the plan command is similar to the output of the diff command that is part of Unix, Linux, and git: anything with a plus sign (+) will be created, anything with a minus sign (−) will be deleted, and anything with a tilde sign (~) will be modified in place.
- In the preceding output, you can see that Terraform is planning on creating a single EC2 Instance and nothing else, which is exactly what you want.

Deploying a Single Server

- To actually create the Instance, run the terraform apply command:
- You'll notice that the apply command shows you the same plan output and asks you to confirm whether you actually want to proceed with this plan.
- Type yes and hit Enter to deploy the EC2 Instance
- Congrats, you've just deployed an EC2 Instance in your AWS account using Terraform!

```
$ terraform apply
```

```
(...)
```

Terraform will perform the following actions:

```
# aws_instance.example will be created
+ resource "aws_instance" "example" {
  + ami                        = "ami-0fb653ca2d3203ac1"
  + arn                       = (known after apply)
  + associate_public_ip_address = (known after apply)
                                pply)
  + cpu_core_count            = (known after apply)
  + cpu_threads_per_core      = (known after apply)
  + get_password_data         = false
  + host_id                   = (known after apply)
  + id                        = (known after apply)
  + instance_state            = (known after apply)
  + instance_type             = "t2.micro"
  + ipv6_address_count        = (known after apply)
  + ipv6_addresses            = (known after apply)
  + key_name                   = (known after apply)
  (...)
}
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

```
Do you want to perform these actions?
```

```
Terraform will perform the actions described above.
```

```
Only 'yes' will be accepted to approve.
```

```
Enter a value:
```

Deploying a Single Server

EC2 Instance doesn't have a name. To add one, you can add tags to the `aws_instance` resource:

```
resource "aws_instance" "example" {  
  ami = "ami-0fb653ca2d3203ac1"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "terraform-example"  
  }  
}
```

Deploying a Single Server

Run terraform apply again to see what this would do:

Terraform says Refreshing state... when you run the apply command), and it can show you a diff between what's currently deployed and what's in your Terraform code (this is one of the advantages of using a declarative language over a procedural one,

```
$ terraform apply
```

```
aws_instance.example: Refreshing state...  
(...)
```

Terraform will perform the following actions:

```
# aws_instance.example will be updated in-place  
~ resource "aws_instance" "example" {  
    ami                               = "ami-0fb653ca2d3203ac1"  
    availability_zone                 = "us-east-2b"  
    instance_state                    = "running"  
    (...)  
+   tags                             = {  
    +   "Name" = "terraform-example"  
    }  
    (...)  
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value:

Deploying a Single Web Server

The next step is to run a web server on this Instance. The goal is to deploy the simplest web architecture possible: a single web server that can respond to HTTP requests

let's run a dirt-simple web server that always returns the text "Hello, World":

```
#!/bin/bash  
echo "Hello, World" > index.xhtml  
nohup busybox httpd -f -p 8080 &
```

This is a Bash script that writes the text "Hello, World" into index.xhtml and runs a tool called busybox (which is installed by default on Ubuntu) to fire up a web server on port 8080 to serve that file.

Deploying a Single Web Server

- Since the dummy web server in this example is just a one-liner that uses busybox, you can use a plain Ubuntu 20.04 AMI and run the “Hello, World” script as part of the EC2 Instance’s User Data configuration.
- When you launch an EC2 Instance, you have the option of passing either a shell script or cloud-init directive to User Data, and the EC2 Instance will execute it during its very first boot.
- You pass a shell script to User Data by setting the `user_data` argument in your Terraform code as follows:

```
resource "aws_instance" "example" {  
  ami           = "ami-0fb653ca2d3203ac1"  
  instance_type = "t2.micro"  
  
  user_data = <<-EOF  
    #!/bin/bash  
    echo "Hello, World" > index.xhtml  
    nohup busybox httpd -f -p 8080 &  
    EOF  
  
  user_data_replace_on_change = true  
  
  tags = {  
    Name = "terraform-example"  
  }  
}
```


Deploying a Single Web Server

Two things to notice about the preceding code:

- The <<-EOF and EOF are Terraform's heredoc syntax, which allows you to create multiline strings without having to insert \n characters all over the place.
- The user_data_replace_on_change parameter is set to true so that when you change the user_data parameter and run apply, Terraform will terminate the original instance and launch a totally new one.
- Terraform's default behavior is to update the original instance in place, but since User Data runs only on the very first boot, and your original instance already went through that boot process, you need to force the creation of a new instance to ensure your new User Data script actually gets executed.

```
resource "aws_instance" "example" {  
  ami           = "ami-0fb653ca2d3203ac1"  
  instance_type = "t2.micro"  
  
  user_data = <<-EOF  
    #!/bin/bash  
    echo "Hello, World" > index.xhtml  
    nohup busybox httpd -f -p 8080 &  
  EOF  
  
  user_data_replace_on_change = true  
  
  tags = {  
    Name = "terraform-example"  
  }  
}
```

Deploying a Single Web Server

- You need to do one more thing before this web server works. By default, AWS does not allow any incoming or outgoing traffic from an EC2 Instance.
- To allow the EC2 Instance to receive traffic on port 8080, you need to create a security group.
- Simply creating a security group isn't enough; you need to tell the EC2 Instance to actually use it by passing the ID of the security group into the `vpc_security_group_ids` argument of the `aws_instance` resource.
- **To do that, you first need to learn about Terraform expressions.**

```
resource "aws_security_group" "instance" {  
  name = "terraform-example-instance"  
  
  ingress {  
    from_port = 8080  
    to_port   = 8080  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

Deploying a Single Web Server

- An expression in Terraform is anything that returns a value.
- One particularly useful type of expression is a **reference**, which allows you to access values from other parts of your code. To access the ID of the security group resource, you are going to need to use a resource attribute reference, which uses the following syntax:

`<PROVIDER>_<TYPE>.<NAME>.<ATTRIBUTE>`

- PROVIDER: is the name of the provider (e.g., aws),
- TYPE is the type of resource (e.g., security_group),
- NAME is the name of that resource (e.g., the security group is named "instance"),
- ATTRIBUTE is either one of the arguments of that resource (e.g., name) or one of the attributes exported by the resource (you can find the list of available attributes in the documentation for each resource).
- The security group exports an attribute called id, so the expression to reference it will look like this: **aws_security_group.instance.id**

Deploying a Single Web Server

You can use this security group ID in the `vpc_security_group_ids` argument of the `aws_instance` as follows.

```
resource "aws_instance" "example" {
  ami           = "ami-0fb653ca2d3203ac1"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]

  user_data = <<-EOF
    #!/bin/bash
    echo "Hello, World" > index.html
    nohup busybox httpd -f -p 8080 &
  EOF

  user_data_replace_on_change = true

  tags = {
    Name = "terraform-example"
  }
}
```

Deploying a Single Web Server

- When you add a reference from one resource to another, you create an implicit dependency.
- Terraform parses these dependencies, builds a dependency graph from them, and uses that to automatically determine in which order it should create resources.
- For example, if you were deploying this code from scratch, Terraform would know that it needs to create the security group before the EC2 Instance,
- because the EC2 Instance references the ID of the security group. You can even get Terraform to show you the dependency graph by running the graph command.

Deploying a Web Server - Variables

- You might have noticed that the web server code has the port 8080 duplicated in both the security group and the User Data configuration.
- This violates the Don't Repeat Yourself (DRY) principle: every piece of knowledge must have a single, unambiguous, authoritative representation within a system.
- If you have the port number in two places, it's easy to update it in one place but forget to make the same change in the other place.
- To allow you to make your code more DRY and more configurable, Terraform allows you to define input variables.

Here's the syntax for declaring a variable:

```
variable "NAME" {  
[CONFIG ...]  
}
```

Deploying a Configurable Web Server

The body of the variable declaration can contain the following optional parameters:

description

It's always a good idea to use this parameter to document how a variable is used.

default

There are a number of ways to provide a value for the variable, including passing it in at the command line (using the `-var` option), via a file (using the `-var-file` option), or via an environment variable (Terraform looks for environment variables of the name `TF_VAR_<variable_name>`). If no value is passed in, the variable will fall back to this default value. If there is no default value, Terraform will interactively prompt the user for one.

Type

This allows you to enforce type constraints on the variables a user passes in. Terraform supports a number of type constraints, including string, number, bool, list, map, set, object, tuple, and any. It's always a good idea to define a type constraint to catch simple errors. If you don't specify a type, Terraform assumes the type is any.

Deploying a Configurable Web Server

validation

This allows you to define custom validation rules for the input variable that go beyond basic type checks, such as enforcing minimum or maximum values on a number.

sensitive

If you set this parameter to true on an input variable, Terraform will not log it when you run plan or apply. You should use this on any secrets you pass into your Terraform code via variables: e.g., passwords, API keys, etc.

Deploying a Configurable Web Server

An example of a variable – type number

```
variable "number_example" {  
  description = "An example of a number variable in Terraform"  
  type = number  
  default = 42  
}
```

And here's an example of a variable - list:

```
variable "list_example" {  
  description = "An example of a list in Terraform"  
  type = list  
  default = ["a", "b", "c"]  
}
```

Deploying a Configurable Web Server

And here's a map that requires all of the values to be strings:

```
variable "map_example" {  
  description = "An example of a map in Terraform"  
  type = map(string)  
  default = {  
    key1 = "value1"  
    key2 = "value2"  
    key3 = "value3"  
  }  
}
```

Deploying a Configurable Web Server

You can also create more complicated structural types using the object type constraint:

```
variable "object_example" {  
  description = "An example of a structural type in Terraform"  
  type = object({  
    name = string  
    age = number  
    tags = list(string)  
    enabled = bool  
  })  
  default = {  
    name = "value1"  
    age = 42  
    tags = ["a", "b", "c"]  
    enabled = true}}
```

Deploying a Configurable Web Server

The preceding example creates an input variable that will require the value to be an object with the keys name (which must be a string), age (which must be a number), tags (which must be a list of strings), and enabled (which must be a Boolean).

If you try to set this variable to a value that doesn't match this type, Terraform immediately gives you a type error. The following example demonstrates trying to set enabled to a string instead of a Boolean

```
variable "object_example_with_error" {
  description = "An example of a structural type in Terraform
with an error"
  type        = object({
    name      = string
    age       = number
    tags      = list(string)
    enabled   = bool
  })

  default = {
    name      = "value1"
    age       = 42
    tags      = ["a", "b", "c"]
    enabled   = "invalid"
  }
}
```

Deploying a Configurable Web Server

The preceding example creates an input variable that will require the value to be an object with the keys name (which must be a string), age (which must be a number), tags (which must be a list of strings), and enabled (which must be a Boolean). If you try to set this variable to a value that doesn't match this type, Terraform immediately gives you a type error. The following example demonstrates trying to set enabled to a string instead of a Boolean

You get the following error:

```
$ terraform apply
```

```
Error: Invalid default value for variable
```

```
    on variables.tf line 78, in variable
"object_example_with_error":
 78:   default = {
 79:     name     = "value1"
 80:     age      = 42
 81:     tags     = ["a", "b", "c"]
 82:     enabled  = "invalid"
 83:   }
```

This default value is not compatible with the variable's type

Deploying a Configurable Web Server

Coming back to the web server example, what you need is a variable that stores the port number:

```
variable "server_port" {  
  description = "The port the server will use for HTTP requests"  
  type = number  
}
```

Note that the `server_port` input variable has no default, so if you run the `apply` command now, Terraform will interactively prompt you to enter a value for `server_port` and show you the description of the variable:

```
$ terraform apply
```

```
var.server_port
```

```
The port the server will use for HTTP requests
```

```
Enter a value:
```

Deploying a Configurable Web Server

If you don't want to deal with an interactive prompt, you can provide a value for the variable via the `-var` command-line option:

```
$ terraform plan -var "server_port=8080"
```

You could also set the variable via an environment variable named `TF_VAR_<name>`, where `<name>` is the name of the variable you're trying to set:

```
$ export TF_VAR_server_port=8080
```

```
$ terraform plan
```

And if you don't want to deal with remembering extra command-line arguments every time you run `plan` or `apply`, you can specify a default value

Deploying a Configurable Web Server

To use the value from an input variable in your Terraform code, you can use a new type of expression called a variable reference, which has the following syntax:

var.<VARIABLE_NAME>

For example, here is how you can set the `from_port` and `to_port` parameters of the security group to the value of the `server_port` variable:

```
resource "aws_security_group" "instance" {  
  name = "terraform-example-instance"  
  ingress {  
    from_port = var.server_port  
    to_port = var.server_port  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```


Deploying a Configurable Web Server

You can put any valid reference within the curly braces, and Terraform will convert it to a string. For example, here's how you can use `var.server_port` inside of the User Data string:

```
user_data = <<-EOF  
#!/bin/bash  
echo "Hello, World" > index.xhtmll  
nohup busybox httpd -f -p ${var.server_port} &  
EOF
```

Deploying a Configurable Web Server

In addition to input variables, Terraform also allows you to define output variables by using the following syntax:

```
output "<NAME>" {  
  value = <VALUE>  
  [CONFIG ...]  
}
```

The NAME is the name of the output variable, and VALUE can be any Terraform expression that you would like to output.

The CONFIG can contain the following optional parameters:

description

It's always a good idea to use this parameter to document what type of data is contained in the output variable.

sensitive

Set this parameter to true to instruct Terraform not to log this output at

Deploying a Configurable Web Server

sensitive

Set this parameter to true to instruct Terraform not to log this output at the end of plan or apply. This is useful if the output variable contains secrets such as passwords or private keys. Note that if your output variable references an input variable or resource attribute marked with sensitive = true, you are required to mark the output variable with sensitive = true as well to indicate you are intentionally outputting a secret.

Deploying a Configurable Web Server

For example, instead of having to manually poke around the EC2 console to find the IP address of your server, you can provide the IP address as an output variable:

```
output "public_ip" {  
  value = aws_instance.example.public_ip  
  description = "The public IP address of the web  
  server"  
}
```

This code uses an attribute reference again, this time referencing the `public_ip` attribute of the `aws_instance` resource. If you run the `apply` command again, Terraform will not apply any changes (because you haven't changed any resources), but it will show you the new output at the very end:

```
$ terraform apply  
  
(...)  
  
aws_security_group.instance: Refreshing state... [id=sg-  
078ccb4f9533d2c1a]  
aws_instance.example: Refreshing state... [id=i-  
028cad2d4e6bddec6]  
  
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
public_ip = "54.174.13.5"
```

Deploying a Configurable Web Server

You can also use the terraform output command to list all outputs without applying any changes:

```
$ terraform output
```

```
public_ip = "54.174.13.5"
```

And you can run terraform output <OUTPUT_NAME> to see the value of a specific output called <OUTPUT_NAME>:

```
$ terraform output public_ip
```

```
"54.174.13.5"
```

Deploying a Cluster of Web Servers

Using an Auto Scaling Group (ASG)

- The first step in creating an ASG is to create a launch configuration, which specifies how to configure each EC2 Instance in the ASG.
- **aws_launch_configuration** resource uses almost the same parameters as the **aws_instance** resource, although it doesn't support tags

```
resource "aws_launch_configuration" "example" {  
    image_id      = "ami-0fb653ca2d3203ac1"  
    instance_type = "t2.micro"  
    security_groups = [aws_security_group.instance.id]  
  
    user_data = <<-EOF  
        #!/bin/bash  
        echo "Hello, World" > index.xhtml  
        nohup busybox httpd -f -p ${var.server_port} &  
        EOF  
}
```

Deploying a Cluster of Web Servers

Now you can create the ASG itself using the `aws_autoscaling_group` resource

Note that the ASG uses a reference to fill in the launch configuration name. This leads to a problem: launch configurations are immutable, so if you change any parameter of your launch configuration, Terraform will try to replace it. Normally, when replacing a resource, Terraform would delete the old resource first and then creates its replacement, but because your ASG now has a reference to the old resource, Terraform won't be able to delete

```
resource "aws_autoscaling_group" "example" {
  launch_configuration = aws_launch_configuration.example.name

  min_size = 2
  max_size = 10

  tag {
    key           = "Name"
    value         = "terraform-asg-example"
    propagate_at_launch = true
  }
}
```

Deploying a Cluster of Web Servers

To solve this problem, you can use a lifecycle setting. Every Terraform resource supports several lifecycle settings that configure how that resource is created, updated, and/or deleted. A particularly useful lifecycle setting is `create_before_destroy`.

If you set `create_before_destroy` to `true`, Terraform will invert the order in which it replaces resources, creating the replacement resource first (including updating any references that were pointing at the old resource to point to the replacement) and then deleting the old resource.

```
resource "aws_launch_configuration" "example" {
  image_id      = "ami-0fb653ca2d3203ac1"
  instance_type = "t2.micro"
  security_groups = [aws_security_group.instance.id]

  user_data = <<-EOF
    #!/bin/bash
    echo "Hello, World" > index.xhtml
    nohup busybox httpd -f -p ${var.server_port} &
  EOF

  # Required when using a launch configuration with an auto
  scaling group.
  lifecycle {
    create_before_destroy = true
  }
}
```


Deploying a Cluster of Web Servers

- There's also one other parameter that you need to add to your ASG to make it work: `subnet_ids`.
- You could hardcode the list of subnets, but that won't be maintainable or portable, so a better option is to use data sources to get the list of subnets in your AWS account.
- A data source represents a piece of read-only information that is fetched from the provider (in this case, AWS) every time you run Terraform.
- Adding a data source to your Terraform configurations does not create anything new; it's just a way to query the provider's APIs for data and to make that data available to the rest of your Terraform code.
- Each Terraform provider exposes a variety of data sources. For example, the AWS Provider includes data sources to look up VPC data, subnet data, AMI IDs, IP address ranges, the current user's identity, and much more.

Deploying a Cluster of Web Servers

- The syntax for using a data source is very similar to the syntax of a resource:

```
data "<PROVIDER>_<TYPE>" "<NAME>" {  
[CONFIG ...]  
}
```

- Here, PROVIDER is the name of a provider (e.g., aws), TYPE is the type of data source you want to use (e.g., vpc), NAME is an identifier you can use throughout the Terraform code to refer to this data source, and CONFIG consists of one or more arguments that are specific to that data source.

Deploying a Cluster of Web Servers

- For example, here is how you can use the `aws_vpc` data source to look up the data for your Default VPC

```
data "aws_vpc" "default" {  
  default = true  
}
```

- Note that with data sources, the arguments you pass in are typically search filters that indicate to the data source what information you're looking for.
- With the `aws_vpc` data source, the only filter you need is `default = true`, which directs Terraform to look up the Default VPC in your AWS account.
- To get the data out of a data source, you use the following attribute reference syntax:

```
data.<PROVIDER>_<TYPE>.<NAME>.<ATTRIBUTE>
```

Deploying a Cluster of Web Servers

- For example, to get the ID of the VPC from the `aws_vpc` data source, you would use the following:
`data.aws_vpc.default.id`
- You can combine this with another data source, `aws_subnets`, to look up the subnets within that VPC:

```
data "aws_subnets" "default" {  
  filter {  
    name = "vpc-id"  
    values = [data.aws_vpc.default.id]  
  }  
}
```

Deploying a Cluster of Web Servers

- Finally, you can pull the subnet IDs out of the `aws_subnets` data source and tell your ASG to use those subnets via the `vpc_zone_identifier` argument:

```
resource "aws_autoscaling_group" "example" {  
  launch_configuration = aws_launch_configuration.example.name  
  vpc_zone_identifier = data.aws_subnets.default.ids  
  min_size = 2  
  max_size = 10  
  tag {  
    key = "Name"  
    value = "terraform-asg-example"  
    propagate_at_launch = true  
  }  
}
```

Deploying a Cluster of Web Servers

Add a LB

- The first step is to create the ALB itself using the `aws_lb` resource:

```
resource "aws_lb" "example" {  
  name = "terraform-asg-example"  
  load_balancer_type = "application"  
  subnets = data.aws_subnets.default.ids  
}
```

- Note that the `subnets` parameter configures the load balancer to use all the subnets in your Default VPC by using the `aws_subnets` data source

Deploying a Cluster of Web Servers

- The next step is to define a listener for this ALB using the `aws_lb_listener` resource:

```
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.example.arn
  port              = 80
  protocol          = "HTTP"

  # By default, return a simple 404 page
  default_action {
    type = "fixed-response"

    fixed_response {
      content_type = "text/plain"
      message_body = "404: page not found"
      status_code  = 404
    }
  }
}
```

Deploying a Cluster of Web Servers

- This listener configures the ALB to listen on the default HTTP port, port 80, use HTTP as the protocol, and send a simple 404 page as the default response for requests that don't match any listener rules.
- Note that, by default, all AWS resources, including ALBs, don't allow any incoming or outgoing traffic, so you need to create a new security group specifically for the ALB. This security group should allow incoming requests on port 80 so that you can access the load balancer over HTTP, and allow outgoing requests on all ports so that the load balancer can perform health checks:

```
resource "aws_security_group" "alb" {  
    name = "terraform-example-alb"  
  
    # Allow inbound HTTP requests  
    ingress {  
        from_port = 80  
        to_port   = 80  
        protocol  = "tcp"  
        cidr_blocks = ["0.0.0.0/0"]  
    }  
  
    # Allow all outbound requests  
    egress {  
        from_port = 0  
        to_port   = 0  
        protocol  = "-1"  
        cidr_blocks = ["0.0.0.0/0"]  
    }  
}
```


Deploying a Cluster of Web Servers

- You'll need to tell the `aws_lb` resource to use this security group via the `security_groups` argument:

```
resource "aws_lb" "example" {  
  name                = "terraform-asg-example"  
  load_balancer_type = "application"  
  subnets            = data.aws_subnets.default.ids  
  security_groups     = [aws_security_group.alb.id]  
}
```

Deploying a Cluster of Web Servers

- Next, you need to create a target group for your ASG using the `aws_lb_target_group` resource
- This target group will health check your Instances by periodically sending an HTTP request to each Instance and will consider the Instance “healthy” only if the Instance returns a response that matches the configured matcher

```
resource "aws_lb_target_group" "asg" {  
    name      = "terraform-asg-example"  
    port      = var.server_port  
    protocol  = "HTTP"  
    vpc_id    = data.aws_vpc.default.id  
  
    health_check {  
        path                = "/"  
        protocol             = "HTTP"  
        matcher              = "200"  
        interval             = 15  
        timeout              = 3  
        healthy_threshold    = 2  
        unhealthy_threshold  = 2  
    }  
}
```

Deploying a Cluster of Web Servers

- How does the target group know which EC2 Instances to send requests to?
- You could attach a static list of EC2 Instances to the target group using the `aws_lb_target_group_attachment` resource, but with an ASG, Instances can launch or terminate at any time, so a static list won't work.
- Instead, you can take advantage of the first-class integration between the ASG and the ALB. Go back to the `aws_autoscaling_group` resource, and set its `target_group_arns` argument to point at your new target group:

```
resource "aws_autoscaling_group" "example" {
  launch_configuration = aws_launch_configuration.example.name
  vpc_zone_identifier  = data.aws_subnets.default.ids

  target_group_arns = [aws_lb_target_group.asg.arn]
  health_check_type = "ELB"

  min_size = 2
  max_size = 10

  tag {
    key           = "Name"
    value         = "terraform-asg-example"
    propagate_at_launch = true
  }
}
```

Deploying a Cluster of Web Servers

- How does the target group know which EC2 Instances to send requests to?
- You could attach a static list of EC2 Instances to the target group using the `aws_lb_target_group_attachment` resource, but with an ASG, Instances can launch or terminate at any time, so a static list won't work.
- Instead, you can take advantage of the first-class integration between the ASG and the ALB. Go back to the `aws_autoscaling_group` resource, and set its `target_group_arns` argument to point at your new target group:

```
resource "aws_autoscaling_group" "example" {
  launch_configuration = aws_launch_configuration.example.name
  vpc_zone_identifier  = data.aws_subnets.default.ids

  target_group_arns = [aws_lb_target_group.asg.arn]
  health_check_type = "ELB"

  min_size = 2
  max_size = 10

  tag {
    key           = "Name"
    value         = "terraform-asg-example"
    propagate_at_launch = true
  }
}
```

Deploying a Cluster of Web Servers

- You should also update the `health_check_type` to "ELB".
- The default `health_check_type` is "EC2", which is a minimal health check that considers an Instance unhealthy only if the AWS hypervisor says the VM is completely down or unreachable.
- The "ELB" health check is more robust, because it instructs the ASG to use the target group's health check to determine whether an Instance is healthy and to automatically replace Instances if the target group reports them as unhealthy.:

```
resource "aws_autoscaling_group" "example" {
  launch_configuration = aws_launch_configuration.example.name
  vpc_zone_identifier  = data.aws_subnets.default.ids

  target_group_arns = [aws_lb_target_group.asg.arn]
  health_check_type = "ELB"

  min_size = 2
  max_size = 10

  tag {
    key           = "Name"
    value         = "terraform-asg-example"
    propagate_at_launch = true
  }
}
```

Deploying a Cluster of Web Servers

- Finally, it's time to tie all these pieces together by creating listener rules using the `aws_lb_listener_rule` resource:

```
resource "aws_lb_listener_rule" "asg" {
  listener_arn = aws_lb_listener.http.arn
  priority     = 100

  condition {
    path_pattern {
      values = ["*"]
    }
  }

  action {
    type = "forward"
    target_group_arn = aws_lb_target_group.asg.arn
  }
}
```

Deploying a Cluster of Web Servers

- code adds a listener rule that sends requests that match any path to the target group that contains your ASG.
- There's one last thing to do before you deploy the load balancer—replace the old `public_ip` output of the single EC2 Instance you had before with an output that shows the DNS name of the ALB

```
output "alb_dns_name" {  
    value      = aws_lb.example.dns_name  
    description = "The domain name of the load balancer"  
}
```